

Static Loading and Tidal Love Numbers in DSpecM1D

Alex Myhill

July 29, 2026

This report describes the physical formulation, finite-element implementation, user interfaces, and validation evidence for the static spheroidal Love-number module. External programs are treated as independent comparisons, not as ground truth.

Contents

1	Executive summary	1
2	Using the module	1
2.1	CMake configuration	1
2.2	Command line	2
2.3	C++ API	2
3	Mathematical formulation	3
3.1	Assumptions and generalized responses	3
3.2	Radial fields and topology	3
3.3	Solid weak operator	4
3.4	Internal fluids and interfaces	4
3.5	Forcing and exceptional degrees	5
4	Numerical implementation	5
4.1	Model, mesh, and finite-element space	5
4.2	Corrected background gravity	6
5	Validation and benchmarking	6
5.1	Evidence hierarchy and reproducibility	6
5.2	Analytic gravity and controlled models	7
5.3	Isotropized PREM	8
5.4	TI-PREM and the gravitational constant	8
5.5	External provenance and limitations	10
	Appendix A: Love-number test inventory	12

1 Executive summary

The DSpecM1D Love-number module computes the static, self-gravitating spheroidal response of a spherically symmetric, radially transversely isotropic Earth model. It supports all-solid structures and arbitrary internal fluid layers. The outermost layer must be solid: a surface ocean is rejected because the required free-surface fluid treatment is outside the implemented formulation. Attenuation and rotation are absent.

For each spherical-harmonic degree from zero to a user-selected maximum, the module solves three independent problems: a unit displacement/traction load, a unit gravitational-potential load, and a unit tidal potential. It reports the dimensional generalized responses

$$h_u, \quad k_u, \quad h_\phi, \quad k_\phi, \quad h_t, \quad k_t,$$

and the physical loading sums

$$h_{\text{load}} = h_u + h_\phi, \quad k_{\text{load}} = k_u + k_\phi.$$

The displacement-type loading quantities $h_u, h_\phi, h_{\text{load}}$ have units $\text{m}^3 \text{kg}^{-1}$; their potential counterparts $k_u, k_\phi, k_{\text{load}}$ have units $\text{m}^4 \text{kg}^{-1} \text{s}^{-2}$. The tidal responses h_t and k_t have units $\text{s}^2 \text{m}^{-1}$ and one, respectively.

Degree zero is solved as a radial U, P problem. A constant degree-zero tidal potential has zero gradient, so its tidal response is exactly zero. For degree one the full centre space is retained, but the surface-potential coefficient is eliminated exactly, imposing $P(a) = 0$. This fixes the translation ambiguity in the chosen geocentric frame; it is not the same post-processing convention used by the ELLN reference software.

Validation proceeds from analytic and algebraic checks to independent inter-code comparisons. Constant-density matrices agree with DSpecM1D's existing radial and spheroidal SEM matrices to roundoff. Analytic, piecewise-density tests show that the corrected main-library background gravity is continuous and accurate to roundoff. On a controlled internal-fluid sphere the maximum DSpecM1D–gia3D response difference falls from 6.3×10^{-6} at eight elements to 3.4×10^{-11} at 256 elements. Sampled isotropized PREM calculations agree at approximately the 10^{-7} level at the finest common model sampling. In a degree-10 TI-PREM comparison, using ELLN's gravitational constant explains 68.12% of the signed h_{10} discrepancy and 64.96% of the signed k_{10} discrepancy. The residual relative difference is about 1.1×10^{-4} ; the exported model agrees with the ELLN definitions to machine precision, so the remaining difference is recorded as unresolved rather than attributed to either implementation.

2 Using the module

2.1 CMake configuration

The module is disabled by default. Enable it when configuring DSpecM1D:

```
cmake -S . -B build -DDSPECM1D_BUILD_LOVE_NUMBERS=ON
cmake --build build --parallel
```

With tests enabled, the same build registers the normal Love-number test suite. The external-reference tests are separately gated by `DSPECM1D_ENABLE_PAPER_VALIDATION`.

2.2 Command line

The installed build-tree executable has the positional interface

```
dspecm1d-love MODEL_FILE LMAX OUTPUT_FILE \  
[POLYNOMIAL_ORDER] [MAXIMUM_RADIAL_STEP]
```

The optional polynomial degree defaults to $p = 6$, hence seven Gauss–Lobatto–Legendre (GLL) nodes per element. The default maximum relative radial step is 0.01. An output path of `-` writes to standard output. Configuration and model errors produce a non-zero exit status; in particular, a model whose outermost layer is fluid is rejected.

For example, a short no-ocean PREM calculation can be written to standard output with

```
build/bin/dspecm1d-love data/models/prem.200.no.noatten.txt 2 - 4 0.02
```

The metadata header and first data row have the form

```
# DSpecM1D static Love numbers  
# maximum_degree: 2  
# polynomial_order: 4  
# degree_one_convention: P(a) = 0  
# columns: l h_u k_u h_phi k_phi h_load k_load h_t k_t  
0 -7.2078025353377543e-05 2.5739078430898041e-16 ...
```

Every data row contains nine whitespace-separated columns:

$$l \mid h_u \quad k_u \quad h_\phi \quad k_\phi \quad h_{\text{load}} \quad k_{\text{load}} \quad h_t \quad k_t.$$

The units are those stated in the executive summary and are repeated in the file header. Rows are ordered from $l = 0$ through LMAX.

2.3 C++ API

The public interface is contained in the single header `DSpecM1D/LoveNumbers`. This minimal program uses the exact public types and result members:

```
#include <DSpecM1D/LoveNumbers>  
  
int main() {  
    const EarthModels::ModelInput<double> model(  
        "data/models/prem.200.no.noatten.txt");  
    const DSpecM1D::LoveNumbers::Config config{  
        .maximum_degree = 10,  
        .polynomial_order = 6,  
        .maximum_radial_step = 0.01,  
    };  
  
    const auto results =  
        DSpecM1D::LoveNumbers::calculate(model, config);  
    for (const auto &result : results) {  
        const int degree = result.degree;
```

Table 1: Public response quantities and dimensional units.

Quantity	Meaning	Unit
$h_u, h_\phi, h_{\text{load}}$	surface radial loading response	$\text{m}^3 \text{kg}^{-1}$
$k_u, k_\phi, k_{\text{load}}$	surface potential loading response	$\text{m}^4 \text{kg}^{-1} \text{s}^{-2}$
h_t	radial displacement per unit tidal potential	$\text{s}^2 \text{m}^{-1}$
k_t	potential response per unit tidal potential	1

```

const double h_load = result.h_load();
const double k_load = result.k_load();
(void)degree;
(void)h_load;
(void)k_load;
}
}

```

The returned vector has `maximum_degree + 1` entries in ascending degree order. The individual fields are `h_u`, `k_u`, `h_phi`, `k_phi`, `h_t`, and `k_t`; the two loading sums are member functions.

3 Mathematical formulation

3.1 Assumptions and generalized responses

The reference state is spherical and in hydrostatic equilibrium. Solids may be radially transversely isotropic, described by the five elastic coefficients A, C, F, L, N . Internal inviscid fluid regions are permitted, including a central fluid, but the surface is solid. The calculation is static, linear, non-rotating, self-gravitating, and unattenuated. Only the scalar spheroidal response is considered; horizontal output and toroidal response are not part of this module. These assumptions are consistent with the variational treatment of self-gravitating loading described by Al-Attar and Tromp [1] and its reciprocity extensions [2].

For surface radius a , write the radial displacement and Eulerian potential perturbation of a single harmonic coefficient as

$$u_{lm}(a) = h_{u,l} \zeta_{u,lm} + h_{\phi,l} \zeta_{\phi,lm} + h_{t,l} \psi_{lm}, \quad (1)$$

$$\phi_{lm}(a) = k_{u,l} \zeta_{u,lm} + k_{\phi,l} \zeta_{\phi,lm} + k_{t,l} \psi_{lm}. \quad (2)$$

Here ζ_u denotes the unit displacement/traction forcing, ζ_ϕ the unit surface-potential forcing, and ψ the unit tidal potential. The implementation forms these as three columns and solves them together. Setting the two generalized loading coefficients equal gives

$$h_{\text{load},l} = h_{u,l} + h_{\phi,l}, \quad k_{\text{load},l} = k_{u,l} + k_{\phi,l}. \quad (3)$$

3.2 Radial fields and topology

For $l \geq 1$, each solid region has radial displacement $U(r)$, tangential spheroidal displacement $V(r)$, and potential perturbation $P(r)$. A fluid region has P only. Potential is continuous across every element and material boundary; U, V are continuous only within each contiguous solid run. At a fluid–solid boundary, the solid-side displacements and the shared potential are independent coefficients.

Degree zero uses continuous radial U, P fields in both solid and fluid layers. At the centre the complete finite-element trial space is retained: $U(0), P(0)$ for $l = 0$, and $U(0), V(0), P(0)$ when the centre is solid for $l \geq 1$. No strong-form regularity condition is imposed there.

For degree one, only the surface coefficient $P(a)$ is removed. Surface U, V , every interior potential coefficient, and the complete centre space remain. Omitting a basis coefficient is an exact constraint rather than a penalty; derivatives of all remaining basis functions in the surface element are still assembled.

3.3 Solid weak operator

Let $q = l(l + 1)$, and define the reduced strains

$$D = rU', \quad S = rV' - V + U, \quad T = 2U - qV. \quad (4)$$

Hatted symbols denote test fields. For a solid element and $l \geq 1$, the implemented symmetric bilinear form is

$$\begin{aligned} \mathcal{K}_s = \int & [C\hat{D}\hat{D} + qL\hat{S}\hat{S} + (A - N)\hat{T}\hat{T} + F(\hat{D}\hat{T} + \hat{T}\hat{D}) + q(q - 2)N\hat{V}\hat{V} \\ & + 4\rho(\pi G\rho r - g)r\hat{U}\hat{U} + q\rho gr(\hat{U}\hat{V} + \hat{V}\hat{U}) \\ & + \rho r^2(\hat{U}\hat{P}' + \hat{P}'\hat{U}) + q\rho r(\hat{V}\hat{P} + \hat{P}\hat{V}) \\ & + \frac{r^2\hat{P}'\hat{P}' + q\hat{P}\hat{P}}{4\pi G}] \, dr. \end{aligned} \quad (5)$$

The first line contains the TI elastic energy; the second contains the equilibrium-gravity terms; the third couples displacement and potential; and the last is the gravitational field energy. The reduced variables in Eq. (4) avoid divisions by r at the centre. For $l \geq 2$, matching to the exterior vacuum contributes

$$\mathcal{K}_{\text{vac}} = \frac{a(l + 1)}{4\pi G} \hat{P}(a)P(a). \quad (6)$$

There is no such term for degree one because $P(a)$ is constrained to zero.

The radial degree-zero form is assembled directly:

$$\begin{aligned} \mathcal{K}_0 = \int & \left[Cr^2\hat{U}'\hat{U}' + 2Fr(\hat{U}'\hat{U} + \hat{U}\hat{U}') \right. \\ & + 4\{\rho(\pi G\rho r - g)r + A - N\}\hat{U}\hat{U} \\ & \left. + \rho r^2(\hat{U}\hat{P}' + \hat{P}'\hat{U}) + \frac{r^2\hat{P}'\hat{P}'}{4\pi G} \right] dr + \frac{a}{4\pi G} \hat{P}(a)P(a). \end{aligned} \quad (7)$$

This same form applies in solid and fluid elements for $l = 0$; the potential-only fluid form below is not substituted.

3.4 Internal fluids and interfaces

For $l \geq 1$, fluid elements retain the gravitational field-energy term and add the static stratification contribution

$$\mathcal{K}_f = \int \left[\frac{r^2\hat{P}'\hat{P}' + q\hat{P}\hat{P}}{4\pi G} + \frac{\rho'}{g}r^2\hat{P}\hat{P} \right] dr. \quad (8)$$

Here $\rho' = d\rho/dr$ is the ordinary one-sided derivative of the density spline in that fluid layer. Equation (8) contains a *positive* coefficient $+\rho'/g$; because stable density commonly decreases

outwards, its numerical value is often negative. No additional minus sign is introduced, and density jumps are not represented as volume derivatives. The combined coefficient $r^2\rho'/g$ is set to zero at $r = 0$, avoiding $0/0$.

At a transition radius, let

$$s = \begin{cases} -1, & \text{solid below, fluid above,} \\ +1, & \text{fluid below, solid above.} \end{cases}$$

Using the density ρ_f on the fluid side and displacement U_s on the solid side, the interface term is

$$\mathcal{K}_\Gamma = s\rho_f r^2 \left[g \hat{U}_s U_s + \hat{U}_s P + \hat{P} U_s \right]. \quad (9)$$

The orientation is determined explicitly from the adjacent materials, not from interface parity. Both directions are tested independently.

3.5 Forcing and exceptional degrees

The three load functionals are accumulated first and negated once:

$$KX = -L. \quad (10)$$

The displacement and gravitational columns have surface contributions

$$L_u[\hat{U}] = a^2 g(a) \hat{U}(a), \quad L_\phi[\hat{P}] = a^2 \hat{P}(a). \quad (11)$$

The second expression is absent at $l = 1$, where there is no surface potential test coefficient. For tide, $\psi(r) = (r/a)^l$. Solid elements contribute

$$\int \rho r \psi(r) \{ l \hat{U} + q \hat{V} \} dr, \quad (12)$$

fluid elements contribute $\int (r^2 \rho' / g) \psi \hat{P} dr$, and each interface contributes $s \rho_f r^2 \psi \hat{U}_s$.

At $l = 0$, the tidal potential is constant and its gradient vanishes, so the tidal right-hand side and h_t, k_t are exactly zero. At $l = 1$, the unconstrained static system has a rigid-translation null vector proportional to $U = 1, V = 1, P = -g(r)$. The exact condition $P(a) = 0$ selects a geocentric representative without removing centre coefficients. ELLN instead reports degree-one values after a deformed-centre transformation. Its conventional $k_1 = 0$ therefore cannot be obtained by applying the generic $l \geq 2$ loading conversion to this constrained solution, and degree-one ELLN comparisons are deliberately excluded.

4 Numerical implementation

4.1 Model, mesh, and finite-element space

`EarthModels::ModelInput<double>` reads a layered radial deck and constructs splines for density and elastic properties. `DSpecM1D`'s existing normalization maps length, density, and time to the nondimensional variables used in assembly. `RadialState` owns one `EarthMesh::RadialMesh`, the sampled `MeshModel`, one-sided density derivatives, and access to the `MeshModel` gravity used throughout the Love-number calculation.

The mesh is constructed with the existing `DSpecM1D` relative radial-step algorithm. The public `polynomial_order` is the polynomial degree p , and each element therefore has $p + 1$ GLL nodes.

Material interfaces are represented by adjacent elements with the same boundary radius but distinct layer numbers. Consequently each element samples its own density spline and its own one-sided derivative at a discontinuity.

`LoveDofMap` performs one explicit outward traversal. Potential coefficients are shared across all boundaries. Solid displacement coefficients are shared across solid–solid boundaries, but are neither allocated in a fluid nor shared across a fluid gap. Degree zero interleaves U, P ; a newly encountered solid node for $l \geq 1$ interleaves U, V, P . Degree one differs only by omission of surface P .

Element contributions are inserted directly into an Eigen sparse matrix. For each degree, one `SparseLU` factorization solves the three columns simultaneously. The same map is used for matrix assembly, forcing, and surface extraction. The nondimensional surface values are converted back using `DSpecM1D`’s `LengthNorm`, `DensityNorm`, and `TimeNorm`; the resulting units are listed in Table 1.

4.2 Corrected background gravity

The main `DSpecM1D MeshModel` background-gravity integration was corrected in commit

207b6462ab169a86b7cd4ea183424e2a65d7c34b.

The authoritative calculation integrates enclosed mass:

$$I(r) = \int_0^r \rho(s)s^2 ds, \quad g(r) = \frac{4\pi GI(r)}{r^2}, \quad g(0) = 0. \quad (13)$$

Integration is cumulative from the centre, so enclosed mass and gravity are continuous at every duplicated element and material boundary. Each interval is split at the density-spline knots returned for its model layer, and density is evaluated at the actual quadrature radius.

`RadialState::gravity` and `surfaceGravity` now delegate directly to the owned `MeshModel`. The former duplicate private Love-number integration has been removed, while the operator and forcing continue to use the same `RadialState` accessors. The current production implementation therefore has one authoritative gravity calculation.

5 Validation and benchmarking

5.1 Evidence hierarchy and reproducibility

The validation hierarchy is:

1. focused topology, sign, symmetry, forcing, solve, units, and API tests;
2. analytic gravity tests for homogeneous and piecewise-density spheres;
3. constant-density matrix oracles using `DSpecM1D hR()` and `hS(1)`;
4. matched homogeneous and controlled-fluid comparisons with `gia3D`;
5. a three-way isotropized-PREM comparison with `gia3D`; and
6. a TI-PREM loading comparison with the official ELLN software.

The final two levels are independent comparisons rather than exact regression oracles. The `gia3D` solid matrix is isotropic, so it cannot validate the complete TI operator. Conversely, ELLN reports conventional loading h_l, l_l, k_l , not the separate generalized and tidal responses. The horizontal l_l is not exposed by the `DSpecM1D` public API. The software revisions and publisher package are cited separately from their associated mathematical papers [4–6].

The CSV files behind every figure begin with model, resolution, quantity, and source-script metadata. They are regenerated from labelled validation outputs by `collect_benchmarks.py`; the plotting script contains no Love-number or gravity calculation. Exact commands are listed in `report/README.md`.

5.2 Analytic gravity and controlled models

Figure 1 compares current background gravity on the controlled solid–fluid–solid sphere with its piecewise analytic value. The DSpecM1D error remains between 1.8×10^{-15} and $1.1 \times 10^{-14} \text{ m s}^{-2}$, while gia3D converges towards its roundoff/discretisation floor. An earlier `MeshModel` quadrature defect was identified during Love-number validation and corrected in commit `207b6462ab169a86b7cd4ea183424e2a65d7c34b`. Current Love-number calculations use that corrected shared implementation; the obsolete defective-output curve is no longer retained as an active benchmark.

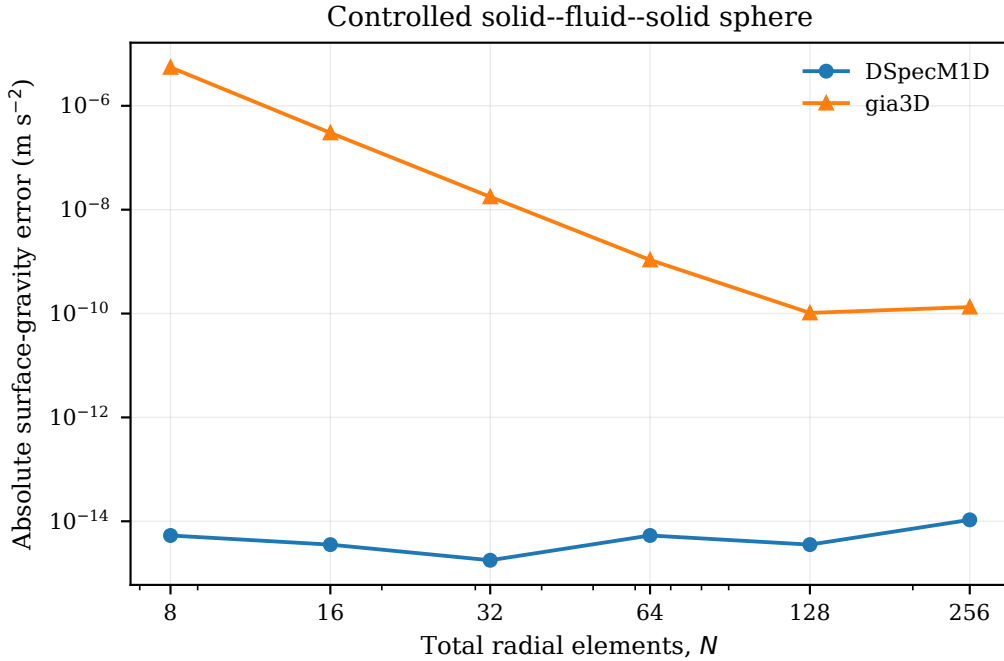


Figure 1: Absolute surface-gravity error for the controlled solid–fluid–solid density profile. DSpecM1D uses the corrected shared `MeshModel` implementation; gia3D is the pinned independent reference. Both errors are measured against the piecewise analytic density integral.

With the corrected gravity, the response comparison converges steadily (Figure 2). At $N = 256$, the maximum component-wise relative differences are 1.45×10^{-11} , 3.37×10^{-11} , and 1.58×10^{-11} for degrees 1, 2, and 10. This is close to the solve-residual scale at that large system size and approaches the homogeneous-sphere solver agreement.

The central-fluid results and all assembled entries remain finite at $r = 0$. Their $N = 32$ inter-code differences are larger than the homogeneous results but still decrease with refinement; no invalid floating-point condition or unintended empty matrix row is observed. Reciprocity is assessed from $g(a)h_\phi - k_u$ in dimensional SI units and is zero for the constrained degree-one columns.

Table 2: Finest controlled-model inter-code diagnostics. The reported difference is the maximum over all eight response components; residuals are the maximum over the three simultaneous solves.

Model	l	N	max. rel. diff.	DSpecM1D residual	DSpec reciprocity
Homogeneous	1	32	7.14×10^{-13}	7.38×10^{-13}	0
	2	32	4.01×10^{-12}	3.99×10^{-12}	2.83×10^{-12}
	10	32	5.72×10^{-13}	1.89×10^{-13}	9.99×10^{-15}
Central fluid	1	32	5.32×10^{-8}	6.11×10^{-13}	0
	2	32	5.06×10^{-8}	6.03×10^{-12}	1.73×10^{-12}
	10	32	3.22×10^{-9}	1.85×10^{-13}	7.04×10^{-13}
Solid–fluid–solid	1	256	1.45×10^{-11}	1.50×10^{-11}	0
	2	256	3.37×10^{-11}	1.32×10^{-10}	7.79×10^{-13}
	10	256	1.58×10^{-11}	3.62×10^{-12}	3.44×10^{-15}

Table 3: Largest component-wise relative difference in each direction for the finest 12.5-km isotropized-PREM deck at radial step 0.0025.

Difference	l	component	relative difference
DSpec sampled – gia3D sampled	10	k_u	$+1.5540 \times 10^{-7}$
gia3D sampled – gia3D analytic	10	k_u	-1.5963×10^{-7}
DSpec sampled – gia3D analytic	50	k_t	-5.2471×10^{-8}

5.3 Isotropized PREM

The isotropized-PREM experiment separates three effects. gia3D first evaluates its analytic PREM representation. The same exporter creates paired isotropic decks, which are then read independently by gia3D and DSpecM1D. Figure 3 shows the maximum response difference over degrees 1, 2, 10, 50 at common radial step 0.0025. The sampled-code difference falls from 2.27×10^{-5} at 100-km knots to 1.55×10^{-7} at 12.5-km knots. The sampled gia3D–analytic PREM difference follows almost the same trend, demonstrating that model interpolation, rather than either linear solve alone, controls the plateau.

At this resolution the sampled DSpecM1D systems have 3728 degrees of freedom for $l = 1$ and 3729 otherwise; gia3D’s sampled and analytic systems have 3616 and 3617. The difference follows their distinct layer meshing while the reported surface radius is common. Maximum three-column residuals are 3.6×10^{-11} , 2.2×10^{-10} , 5.1×10^{-12} , and 1.1×10^{-12} for DSpecM1D degrees 1, 2, 10, and 50. The largest absolute DSpecM1D reciprocity diagnostic is 3.36×10^{-13} . These algebraic errors are below the model-sampling differences.

5.4 TI-PREM and the gravitational constant

The official Chen–Pan–Bevis ELLN calculation [3] supplies an independent TI loading comparison. The ELLN mantle coefficients map directly as

$$A = c_{11}, \quad C = c_{33}, \quad F = c_{13}, \quad L = c_{44}, \quad N = c_{66}.$$

Its analytic layer definitions were sampled into a DSpecM1D deck down to 6.25-km knot spacing. Selected audits on both sides of boundaries found a maximum elastic-coefficient relative mismatch of 6.61×10^{-16} . ELLN uses $G = 6.67384 \times 10^{-11}$ SI, while production DSpecM1D uses 6.67230×10^{-11} SI, a relative difference of 2.30805×10^{-4} .

Matching G explains 68.12% of the signed h_{10} discrepancy and 64.96% of the signed k_{10} discrepancy (Figure 4). The production and matched- G maximum solve residuals are 3.44×10^{-11} and

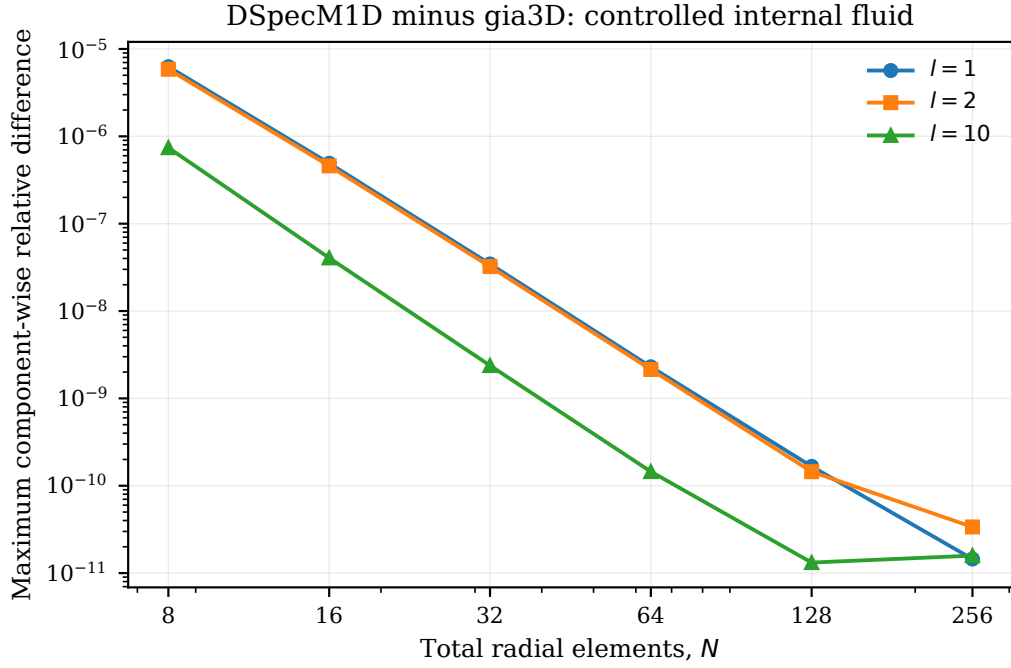


Figure 2: Maximum relative difference over the eight reported response components for matched solid–fluid–solid meshes in DSpecM1D and gia3D. Both interface orientations and a non-zero negative fluid density derivative are present.

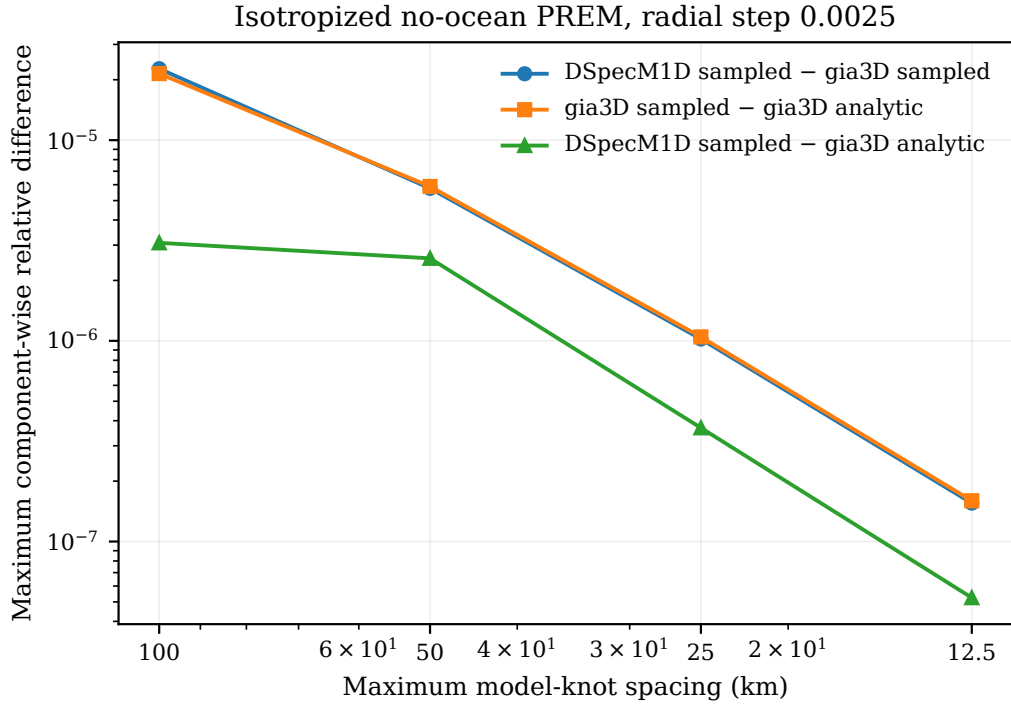


Figure 3: Maximum component-wise relative differences for the three-way isotropized no-ocean PREM comparison at radial step 0.0025. Both sampled calculations approximate, rather than reproduce exactly, gia3D’s analytic PREM representation.

Table 4: Degree-10 comparison with ELLN Table S7 at $p = 6$, maximum radial step 0.00125, and 6.25-km model knots. Signed differences are DSPECM1D minus the published value.

Case	h_{10}	$\Delta h_{10}/ h_{10}^{S7} $	k_{10}	$\Delta k_{10}/ k_{10}^{S7} $
Table S7	-1.451586	—	-0.07066685	—
Production G	-1.45109447	3.3862×10^{-4}	-0.0706430844	3.3630×10^{-4}
ELLN G	-1.45142929	1.0796×10^{-4}	-0.0706585226	1.1784×10^{-4}

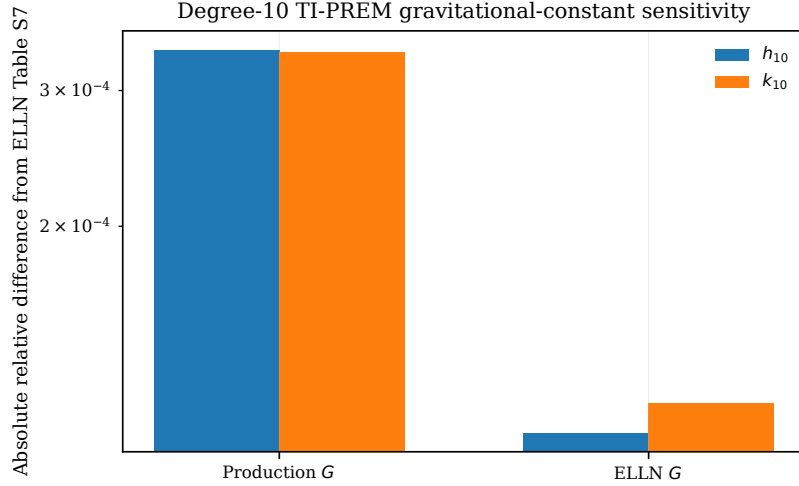


Figure 4: Absolute relative degree-10 discrepancy from ELLN Table S7 using the production gravitational constant and a validation-only source copy patched consistently to ELLN’s constant. The comparison changes only physical G , not the source-tree implementation or model normalization.

3.47×10^{-11} ; absolute reciprocity errors are 3.78×10^{-14} and 4.43×10^{-14} . Thus the remaining 1.1×10^{-4} -scale difference is much larger than the algebraic solve errors but is not explained by a detected model transcription error. Possible representation, tabulation, or inter-code differences remain unresolved. Agreement or disagreement with ELLN alone is not evidence that either calculation is correct or incorrect.

5.5 External provenance and limitations

The paper-validation build copies and patches pinned sources in its build tree only. Two small gia3D patches initialize the fluid quadrature radius before evaluating $r^2 \rho' / g$ and make the central tidal coefficient safe at $r = 0$. A separate ELLN compatibility patch suppresses GUI waitbars for non-interactive reproduction. The gravitational-constant experiment copies the Love-number source to the build tree and changes the one physical constant consistently in gravity, operator, forcing, and conventional conversion paths. None of these changes modifies production source.

Table 5: Pinned external validation inputs. The ELLN archive is not vendored because no redistribution licence was found.

Input	Revision or SHA-256	Role
da380/gia3D	e6a8230fc792416087e530726e3f9f9b41ee0506	isotropic reference
da380/core	eb1d36e418ab0ac9e0b34abb717e6d9b2e863ee5	gia3D support
ggx444_supp.zip	f2904364f9be96239fc85140a032b01b7...	official ELLN package

The published `love.dat` subset is retained as a diagnostic, not an exact oracle: `gia3D` uses analytic PREM coefficients and an isotropic solid matrix, whereas the earlier local deck is sampled and can be TI. ELLN's degree-one frame is not equivalent to $P(a) = 0$, and its horizontal response cannot presently be compared. No external comparison covers a surface ocean, which remains unsupported. Broader formulations, including full-domain gravitation techniques such as those discussed by Yu et al. [7], are outside this static radial module.

Appendix A: Love-number test inventory

The following 69 entries are generated from `love_numbers/TESTS.md`. They correspond exactly to the Love-number registrations in a fresh paper-validation CTest listing: 58 normal tests and 11 gated paper-validation tests. Unrelated core DSpecM1D tests are omitted.

Radial state, material model, and gravity

- `RadialStateTests.PolynomialOrderUsesOneMoreGllNode`
normal; tests/test_radial_state.cpp. Checks that polynomial degree p creates exactly $p+1$ GLL nodes per element.
- `RadialStateTests.MeshEndsAtSurfaceWithoutExteriorBall`
normal; tests/test_radial_state.cpp. Checks that the mesh contains no exterior ball and ends at the model outer radius.
- `RadialStateTests.SampledModelValuesAreFinite`
normal; tests/test_radial_state.cpp. Samples density, TI moduli, gravity, and the density derivative at every element node and requires finite values.
- `RadialStateTests.HomogeneousSphereGravityIsAnalyticAtEveryNode`
normal; tests/test_radial_state.cpp. Compares every nodal gravity value with the homogeneous-sphere analytic solution, including the finite zero at the centre.
- `RadialStateTests.ControlledSolidFluidSolidGravityIsAnalytic`
normal; tests/test_radial_state.cpp. Compares nodal gravity with the piecewise analytic solid-fluid-solid density integral.
- `RadialStateTests.MinimumPolynomialOrderExactlyIntegratesCubicDensityGravity`
normal; tests/test_radial_state.cpp. Checks roundoff-level analytic surface gravity for cubic density at the minimum polynomial order $p=3$.
- `RadialStateTests.ControlledCentralFluidGravityIsAnalytic`
normal; tests/test_radial_state.cpp. Checks the analytic gravity of a constant-density central fluid and solid exterior.
- `RadialStateTests.GravityIsContinuousAcrossEveryBoundary`
normal; tests/test_radial_state.cpp. Requires identical gravity values on both copies of every element and material boundary.
- `RadialStateTests.DensityDerivativeUsesElementLayerSpline`
normal; tests/test_radial_state.cpp. Compares each sampled derivative with the derivative of that element's density spline.
- `RadialStateTests.MaterialInterfaceUsesOneSidedDensityDerivatives`
normal; tests/test_radial_state.cpp. Checks that duplicated-radius interface nodes use the lower and upper layer derivatives independently.
- `RadialStateTests.RejectsSurfaceFluid`
normal; tests/test_radial_state.cpp. Requires a clear exception when the outermost layer is fluid.
- `RadialStateTests.RejectsInvalidConfiguration`
normal; tests/test_radial_state.cpp. Rejects negative maximum degree and non-positive radial step.

DOF mapping

- `LoveDofMapTests.DegreeZeroInterleavesEveryNodeAndMatchesRadialSemMap`
normal; tests/test_love_dof_map.cpp. Checks continuous interleaved U,P indexing and exact agreement with the existing radial SEM map.
- `LoveDofMapTests.DegreeZeroRetainsCentreFields`
normal; tests/test_love_dof_map.cpp. Verifies independent $U(0)$ and $P(0)$ coefficients.
- `LoveDofMapTests.AllSolidDegreeTwoInterleavesEveryField`
normal; tests/test_love_dof_map.cpp. Checks all-solid U,V,P interleaving, dimension, and exact

agreement with the existing spheroidal SEM map.

- `LoveDofMapTests.SolidCentreRetainsThreeIndependentFields`
normal; tests/test_love_dof_map.cpp. Verifies independent $U(0)$, $V(0)$, and $P(0)$ coefficients.
- `LoveDofMapTests.PotentialIsSharedAcrossEveryElementBoundary`
normal; tests/test_love_dof_map.cpp. Checks continuity of the single potential coefficient across solid, fluid, and material boundaries.
- `LoveDofMapTests.DisplacementsAreSharedAcrossSolidBoundaries`
normal; tests/test_love_dof_map.cpp. Checks that adjacent solid elements share their U and V boundary coefficients.
- `LoveDofMapTests.DisplacementsAreAbsentFromFluidElements`
normal; tests/test_love_dof_map.cpp. Requires U and V to return `nullopt` at every fluid node.
- `LoveDofMapTests.InterfacesKeepDisplacementsOnlyOnSolidSide`
normal; tests/test_love_dof_map.cpp. Exercises both interface orientations and checks that displacement exists only on the solid side while P is shared.
- `LoveDofMapTests.InternalFluidDimensionsMatchTopologyFormulas`
normal; tests/test_love_dof_map.cpp. Compares explicit map sizes with the radial-node and separate-solid-run dimension formulas.
- `LoveDofMapTests.DegreeOneOnlyOmitsSurfacePotential`
normal; tests/test_love_dof_map.cpp. Compares degree one and degree two and requires the sole missing coefficient to be surface P .
- `LoveDofMapTests.DegreeOneRetainsSurfaceDisplacements`
normal; tests/test_love_dof_map.cpp. Verifies that degree-one surface U and V remain present.
- `LoveDofMapTests.RejectsNegativeDegree`
normal; tests/test_love_dof_map.cpp. Requires a negative degree to throw `std::invalid_argument`.

All-solid operator

- `StaticOperatorTests.MatchesIsotropicReference`
normal; tests/test_static_operator.cpp. Compares degree 2 and 10 isotropic matrices with `Full1D::SEM::hS(1)`, including dimensions and roundoff symmetry.
- `StaticOperatorTests.MatchesTransverselyIsotropicReference`
normal; tests/test_static_operator.cpp. Repeats the degree 2 and 10 `hS(1)` matrix comparison for a genuinely TI constant-density model.
- `StaticOperatorTests.RetainsCentreFieldsAndMatchesSemOrdering`
normal; tests/test_static_operator.cpp. Checks the three centre coefficients and exact all-solid SEM ordering.
- `StaticOperatorTests.RejectsNegativeDegree`
normal; tests/test_static_operator.cpp. Requires the private static operator to reject a negative degree.

Fluid operator and interfaces

- `StaticFluidOperatorTests.InterfaceSignsUseFluidSideDensity`
normal; tests/test_static_fluids.cpp. Isolates both solid–fluid interface orientations and verifies sign, fluid-side density, shared P , and solid-side U .
- `StaticFluidOperatorTests.NonconstantFluidBlockUsesPositiveDensityGradient`
normal; tests/test_static_fluids.cpp. Compares the fluid potential block with an independent quadrature assembly containing $+r^2 \rho'/g$.
- `StaticFluidOperatorTests.ConstantDensityCentralFluidIsFinite`
normal; tests/test_static_fluids.cpp. Checks zero stratification, a finite central-fluid matrix, and safe handling of $r=g=0$.
- `StaticFluidOperatorTests.InternalFluidMatricesAreCompleteAndSymmetric`

normal; tests/test_static_fluids.cpp. Requires internal-fluid and no-ocean PREM matrices to have the map dimension, roundoff symmetry, and no empty row or column.

Forcing, solve, residuals, and reciprocity

- `StaticForcingTests.SurfaceLoadsHaveExpectedEntriesAndSigns`
normal; tests/test_static_solve.cpp. Checks the displacement and gravitational surface load entries after the single $KX=-L$ negation.
- `StaticForcingTests.AllSolidTideMatchesIndependentAssembly`
normal; tests/test_static_solve.cpp. Compares the all-solid tidal column with a test-local GLL assembly at degrees 2 and 10.
- `StaticForcingTests.InternalFluidTideMatchesBothInterfaceOrientations`
normal; tests/test_static_solve.cpp. Independently assembles solid, fluid-gradient, and both oriented interface tidal terms.
- `StaticSolveTests.OneFactorizationSolvesAllColumnsAndExtractsResults`
normal; tests/test_static_solve.cpp. Checks three-column residuals, combined-load linearity, dimensional extraction, reciprocity, and finite results for isotropic, TI, fluid, and PREM models.
- `StaticSolveTests.RejectsNegativeDegree`
normal; tests/test_static_solve.cpp. Requires the private solve function to reject a negative degree.

Degree zero

- `StaticRadialOperatorTests.MatchesRadialSemForSupportedModels`
normal; tests/test_static_radial.cpp. Compares degree-zero matrices with `Full1D::SEM::hR()` for constant-density isotropic and TI models, with symmetry and ordering checks.
- `StaticRadialOperatorTests.RetainsCentreFields`
normal; tests/test_static_radial.cpp. Verifies independent radial $U(0)$ and $P(0)$ coefficients.
- `StaticRadialForcingTests.SurfaceLoadsAndZeroTideAreExact`
normal; tests/test_static_radial.cpp. Checks the two surface loads and requires an identically zero degree-zero tidal column.
- `StaticRadialSolveTests.SolvesSupportedModels`
normal; tests/test_static_radial.cpp. Checks degree-zero residuals, reciprocity, finite model coverage, and exact zero h_t, k_t .

Degree one

- `StaticDegreeOneOperatorTests.AllSolidMatricesMatchConstrainedSem`
normal; tests/test_static_degree_one.cpp. Compares isotropic and TI matrices with the principal submatrix of $hS(1)$ after removing surface P , and evaluates the full translation null vector.
- `StaticDegreeOneOperatorTests.RetainsCentreAndSurfaceFields`
normal; tests/test_static_degree_one.cpp. Checks independent centre U, V, P , retained surface U, V , and absent surface P .
- `StaticDegreeOneOperatorTests.InternalFluidMatricesHaveExpectedSizeAndSymmetry`
normal; tests/test_static_degree_one.cpp. Checks map dimension and roundoff symmetry for an internal fluid and no-ocean PREM.
- `StaticDegreeOneForcingTests.SurfaceLoadsAndIndependentTidesAreCorrect`
normal; tests/test_static_degree_one.cpp. Verifies the displacement load, exactly zero gravitational column, and independent all-solid and fluid tidal assemblies.
- `StaticDegreeOneSolveTests.SolvesSupportedModels`
normal; tests/test_static_degree_one.cpp. Checks constrained residuals, finite remaining responses, and exact zero potential-related outputs.

Public API

- `LoveNumbersTests.LoadSumsCombineGeneralizedResponses`
normal; tests/test_love_numbers.cpp. Checks that `h_load()` and `k_load()` return the sums of their generalized components.
- `LoveNumbersTests.PublicHeaderCalculatesOrderedDegreeRange`
normal; tests/test_love_numbers.cpp. Uses only the public header and checks an ordered result for every degree from zero through the configured maximum.
- `LoveNumbersTests.MaximumDegreeZeroIsValid`
normal; tests/test_love_numbers.cpp. Checks that a degree-zero-only public calculation succeeds and returns one result.
- `LoveNumbersTests.SupportedModelsReturnFiniteValues`
normal; tests/test_love_numbers.cpp. Runs the public API on isotropic, TI, internal-fluid, and no-ocean PREM models and checks special-degree zeros and finite values.
- `LoveNumbersTests.RejectsInvalidConfigurationAndSurfaceFluid`
normal; tests/test_love_numbers.cpp. Checks public rejection of invalid configuration values and a surface fluid.
- `LoveNumbersTests.EnforcesMinimumPolynomialOrder`
normal; tests/test_love_numbers.cpp. Requires the public calculation to reject polynomial orders below three and accept $p=3$.
- `LoveNumbersPublicPrivateTests.PublicResultsMatchPrivateSolves`
normal; tests/test_public_private.cpp. Compares every public component directly with private solves at selected degrees including 0, 1, 2, and 10.

CLI

- `LoveNumbersCliTests.ExplicitOptionsWriteFileAndMatchPublicResults`
normal; tests/test_cli.cpp. Runs explicit optional arguments, parses the file metadata and nine columns, and compares every value with `calculate`.
- `LoveNumbersCliTests.DefaultsWriteStandardOutput`
normal; tests/test_cli.cpp. Runs the default polynomial order and radial step through standard output and checks ordered rows.
- `LoveNumbersCliTests.InvalidArgumentsAndMissingModelFail`
normal; tests/test_cli.cpp. Requires non-zero status and clear diagnostics for malformed arguments and a missing model.
- `LoveNumbersCliTests.RejectsUnderintegratedPolynomialOrders`
normal; tests/test_cli.cpp. Requires non-zero status and the $p \geq 3$ diagnostic for $p=1$ and $p=2$.
- `LoveNumbersCliTests.SurfaceFluidIsRejectedClearly`
normal; tests/test_cli.cpp. Runs the CLI on a surface-fluid model and checks the explicit no-ocean error.

gia3D validation

- `LoveNumbersGia3DReference.ReportsDirectComparison`
paper validation; validation/gia3d/compare_reference.py. Runs the pinned original-truncation PREM driver, reports three independent solutions and stock combined-load agreement, and requires finite residuals and degree-one zeros.
- `LoveNumbersGia3DReference.ReportsControlledSphereComparison`
paper validation; validation/compare_controlled_sphere.py. Compares matched homogeneous-sphere meshes, dimensions, gravity, residuals, reciprocity, and eight response components.
- `LoveNumbersGia3DReference.ReportsSolidFluidSolidComparison`
paper validation; validation/compare_controlled_sphere.py. Repeats the matched comparison for two interface orientations and a non-zero fluid density derivative, now through $N=256$.
- `LoveNumbersGia3DReference.ReportsCentralFluidComparison`

paper validation; validation/compare_controlled_sphere.py. Checks a central fluid for finite matrices and responses, zero stratification, matching topology, and no invalid floating-point flags.

- `LoveNumbersGia3DReference.ReportsIsotropicPremComparison`
paper validation; validation/compare_isotropic_prem.py. Performs the analytic-gia3D, sampled-gia3D, and sampled-DSpecM1D PREM comparison across mesh and knot refinements.

ELLN validation

- `LoveNumbersELLNReference.ValidatesConversion`
paper validation; validation/elln/validate_elln.py. Algebraically round-trips the dimensional-to-conventional `h_l,k_l` conversions and their signs.
- `LoveNumbersELLNReference.AuditsOfficialExample`
paper validation; validation/elln/validate_elln.py. Verifies the official archive, source and model checksums, runs the non-GUI example when possible, and otherwise reports the publisher table.
- `LoveNumbersELLNReference.AuditsDegreeOneConvention`
paper validation; validation/elln/validate_elln.py. Traces ELLN’s deformed-centre subtraction and demonstrates why it is not the $P(a)=0$ generic conversion.
- `LoveNumbersELLNReference.ReportsTiPremComparison`
paper validation; validation/elln/compare_ti_prem.py. Validates generated TI decks and reports degree-10 Table S7 differences, mesh/knot convergence, residuals, and the differing gravitational constants.
- `LoveNumbersELLNReference.ReportsGravitationalConstantSensitivity`
paper validation; validation/elln/compare_gravitational_constants.py. Compares production and ELLN G , reports sensitivity and explained discrepancy, and audits exported `rho,A,C,F,L,N` against the official model.

Other gated diagnostics

- `LoveNumbersReferenceValidation.ReportsPremConvergence`
paper validation; tests/test_reference_validation.cpp. Reports selected-degree convergence against the published `love.dat` subset without treating the sampled-model comparison as a regression oracle.

References

- [1] David Al-Attar and Jeroen Tromp. Sensitivity kernels for viscoelastic loading based on adjoint methods. *Geophysical Journal International*, 196(1):34–77, 2014. doi: 10.1093/gji/ggt395.
- [2] David Al-Attar, Frank Syvret, Ophelia Crawford, Jerry X. Mitrovica, and Andrew J. Lloyd. Reciprocity and sensitivity kernels for sea level fingerprints. *Geophysical Journal International*, 236(1):362–378, 2024. doi: 10.1093/gji/ggad434.
- [3] J. Y. Chen, E. Pan, and M. Bevis. Accurate computation of the elastic load Love numbers to high spectral degree for a finely layered, transversely isotropic and self-gravitating Earth. *Geophysical Journal International*, 212(2):827–838, 2018. doi: 10.1093/gji/ggx444.
- [4] J. Y. Chen, E. Pan, and M. Bevis. ELLN supporting information for the elastic load Love-number calculation. Supporting information for <https://doi.org/10.1093/gji/ggx444>, 2018. Publisher archive `ggx444_supp.zip`, SHA-256 `f2904364f9be96239fc85140a032b01b732cba6ea1e9bc323f2f3325834e7da7`.
- [5] gia3D project. da380/core numerical and spherical-model support library. <https://github.com/da380/core>, n.d.. Validation revision `eb1d36e418ab0ac9e0b34abb717e6d9b2e863ee5`.
- [6] gia3D project. gia3D static loading and tidal reference software. <https://github.com/da380/gia3D>, n.d.. Validation revision `e6a8230fc792416087e530726e3f9f9b41ee0506`.

- [7] Ziheng Yu, Alex D. C. Myhill, and David Al-Attar. Efficient parallel finite-element methods for planetary gravitation: DtN and multipole expansions, 2026. URL <https://arxiv.org/abs/2602.12075>.